Chapter 7: Collections and Strategy, Iterator Patterns

Collections and Strategy, Iterator Patterns

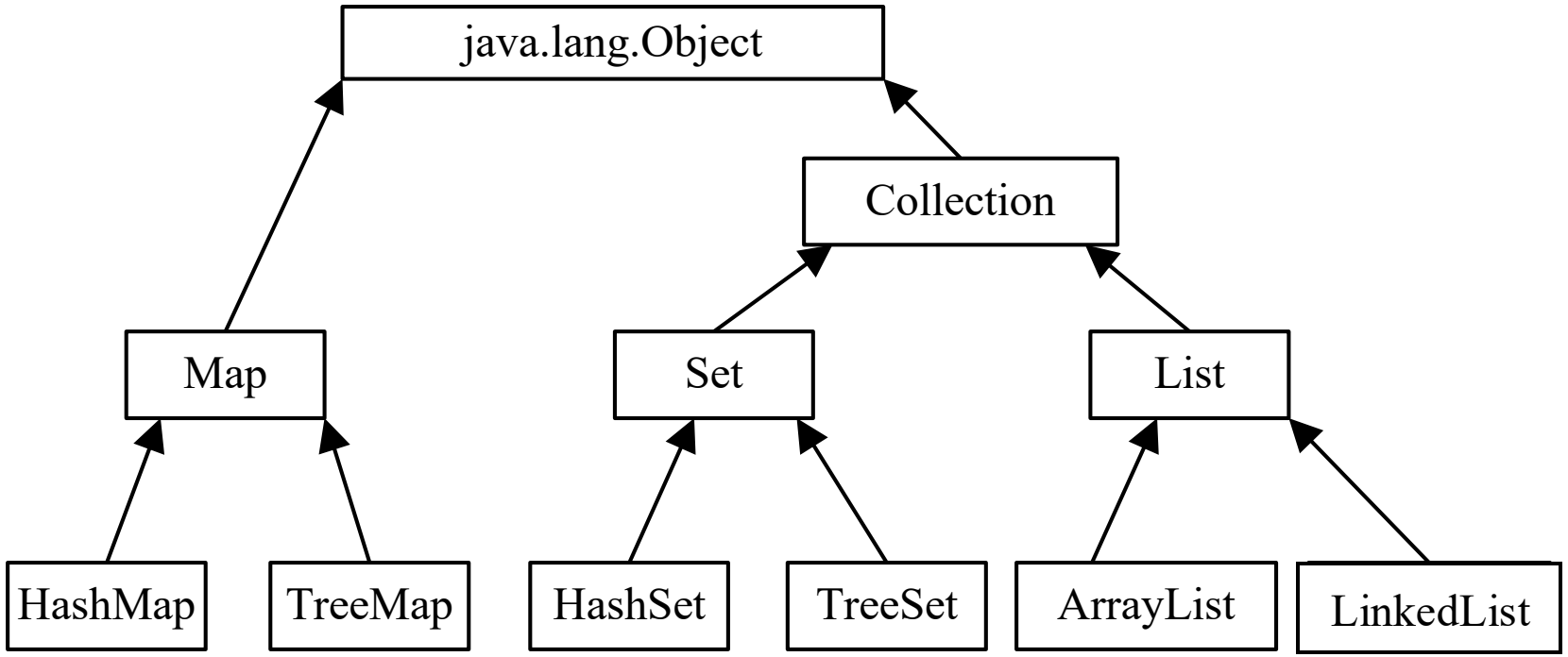
- **Overview of collection classes**
- **List interface and its standard implementation**
- **ArrayList and LinkedList**
- **Set interfaces with Map**
- **Strategy pattern**
- **Iterator pattern**

What is a collection?

- The Java language java.util package provides collection classes, which are also called containers. When it comes to containers, it is inevitable to think of arrays, and the difference between set classes and arrays is that the length of the array is fixed and the length of the collection is variable.
- The interfaces in the collection determine the basic characteristics of each class in the collection API. Specific classes simply provide different implementations of standard interfaces.

What is a collection?

- The most basic interface is the Collection interface, which defines the basic methods for manipulating data.
- Commonly used collections include List collection, Set collection, and Map collection, where List and Set implement the Collection interface. Each interface also provides different implementation classes.



Collections and Strategies, Iterator Patterns

- **Overview of collection classes**
- **List interface and its standard implementation**
- **ArrayList and LinkedList**
- **Set interfaces with Map**
- **Strategy pattern**
- **Iterator pattern**

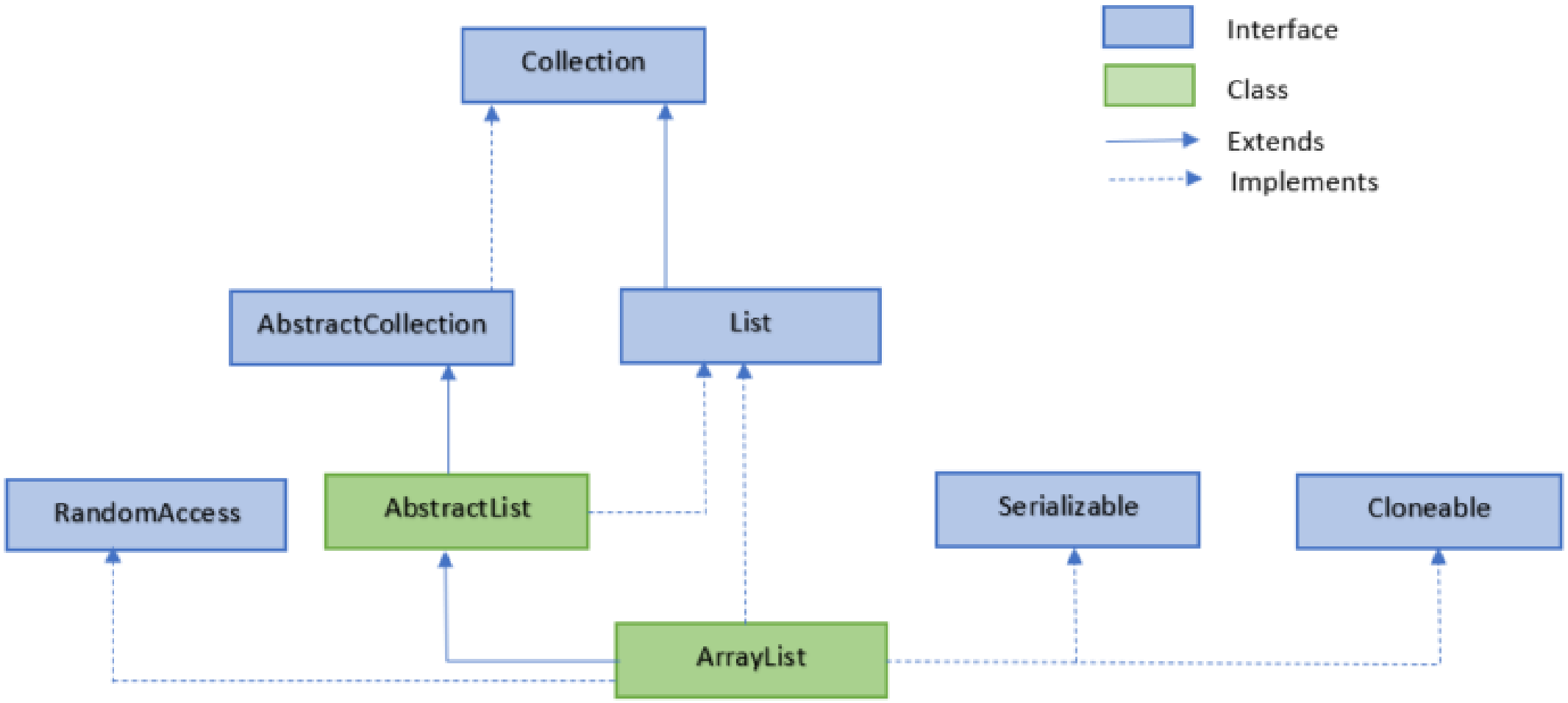
List interface

- The List interface defines an ordered collection of objects that allow duplicate elements to exist.
- Lists are similar to dynamic arrays or variable-length arrays, and the number of elements stored in the List (the capacity of the List) can be automatically adjusted with insertion operations.

ArrayList class

- The ArrayList class is an array that can be dynamically modified, and the difference from ordinary arrays is that it has no fixed size limit, and we can add or remove elements.
- ArrayList inherits from AbstractList and implements the List interface.

ArrayList class



ArrayList class

The ArrayList class is located in the java.util package, and it needs to be imported before use, and the syntax format is as follows:

```
import java.util.ArrayList;
ArrayList<E> objectName =new ArrayList<E>();
```

E: Generic data type, used to set the data type of objectName.

ArrayList is an array queue that provides related functions such as adding, deleting, modifying, traversing, etc.

ArrayList class

```
import java.util.ArrayList;

public class ArrayListTest {
    public static void main(String[] args) {
        ArrayList<String> sites = new ArrayList<String>();
        sites.add("Google");
        sites.add("Amazon");
        sites.add("Taobao");
        sites.add("Weibo");
        System.out.println(sites);
    }
}
```

The output is: [Google, Amazon, Taobao, Weibo]

ArrayList class

Add elements in the middle of the list:

```
import java.util.ArrayList;

public class ArrayListTest {
    public static void main(String[] args) {
        ArrayList<String> sites = new ArrayList<String>();
        sites.add(“Google”);
        sites.add(“Amazon”);
        sites.add(“Taobao”);
        sites.add(1, "Weibo");
        System.out.println(sites);
    }
}
```

The output is： [Google, Weibo, Amazon, Taobao]

ArrayList class

Delete Element: use the remove() method

```
import java.util.ArrayList;

public class ArrayListTest {
    public static void main(String[] args) {
        ArrayList<String> sites = new ArrayList<String>();
        sites.add("Google");
        sites.add("Amazon");
        sites.add("Taobao");
        sites.add("Weibo");
        sites.remove(3); // Remove the element
        System.out.println(sites);
    }
}
```

The output is: [Google, Amazon, Taobao]

ArrayList class

Modify Elements: use the set() method

```
import java.util.ArrayList;

public class ArrayListTest {
    public static void main(String[] args) {
        ArrayList<String> sites = new ArrayList<String>();
        sites.add("Google");
        sites.add("Amazon");
        sites.add("Taobao");
        sites.add("Weibo");
        sites.set(2, "Wiki"); // The first is the index position, and the second is the value
System.out.println(sites);
    }
}
```

The output is: [Google, Amazon, Wiki, Weibo]

ArrayList class

Access elements: use the get() method

```
import java.util.ArrayList;

public class ArrayListTest {
    public static void main(String[] args) {
        ArrayList<String> sites = new ArrayList<String>();
        sites.add("Google");
        sites.add(“Amazon");
        sites.add("Taobao");
        sites.add("Weibo");
        System.out.println(sites.get(1));
    }
}
```

The output is: Amazon

ArrayList类

Calculate size: If you want to calculate the number of elements in an ArrayList, you can use the size() method

```
import java.util.ArrayList;

public class ArrayListTest {
   public static void main(String[] args) {
      ArrayList<String> sites = new ArrayList<String>();
      sites.add("Google");
      sites.add(“Amazon");
      sites.add("Taobao");
      sites.add("Weibo");
      System.out.println(sites.size());
   }
}
```

The output is: 4

ArrayList class

Iterate over the array list: can use for to iterate over elements in an array list

```
import java.util.ArrayList;
public class ArrayListTest {
   public static void main(String[] args) {
      ArrayList<String> sites = new ArrayList<String>();
      sites.add("Google");
      sites.add(‘‘Amazon");
      sites.add("Taobao");
      sites.add("Weibo");
        for (int i = 0; i < sites.size(); i++) {
          System.out.println(sites.get(i));
        }
   }
}
```

The output is:

```
Google
Amazon
Taobao
Weibo
```

ArrayList class

Iterate over array lists: You can also use for-each to iterate over elements

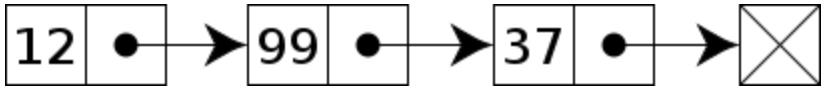
```
import java.util.ArrayList;
public class ArrayListTest {
   public static void main(String[] args) {
      ArrayList<String> sites = new ArrayList<String>();
      sites.add("Google");
      sites.add(“Amazon");
      sites.add("Taobao");
      sites.add("Weibo");
        for (String i : sites) {
          System.out.println(i);
      }
   }
}
```

The output is:

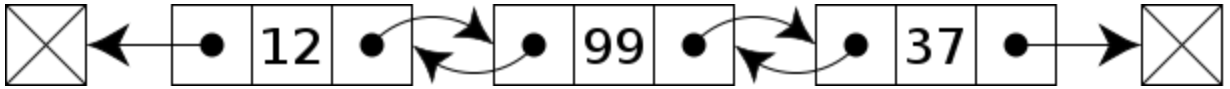
```
Google
Amazon
Taobao
Weibo
```

LinkedList class

- Linked list is a common underlying data structure, a linear table, but it does not store data in a linear order, but stores the address of the next node in each node. Linked lists can be divided into one-way linked lists and two-way linked lists.
- A one-way linked list contains two values: the value of the current node and a link to the next node.



- A bidirectional linked list has three integer values: numeric, backward node link, forward node link.



LinkedList class

- Java LinkedList, similar to ArrayList, is a commonly used data container.
- Compared to ArrayList, LinkedList is more efficient for adding and deleting operations, while finding and modifying operations are less efficient. A one-way linked list contains two values: the value of the current node and a link to the next node.

LinkedList class

Use ArrayList in the following cases:

- Frequent visits to an element in the list.
- Only add and remove elements are required at the end of the list.

Use LinkedList in the following cases:

- Frequent actions to add and remove elements at the beginning, middle, or specified location of the list are required.

LinkedList class

The LinkedList class is located in the java.util package, and it needs to be imported before use, and the syntax format is as follows:

```
import java.util.LinkedList;
LinkedList<E> objectName = new LinkedList<E>(); // Constructor
```

E: Generic data type, used to set the data type of objectName.

LinkedList class

```
import java.util.LinkedList;

public class LinkedListTest {
    public static void main(String[] args) {
        LinkedList<String> sites = new LinkedList<String>();
        sites.add("Google");
        sites.add("Amazon");
        sites.add("Taobao");
        sites.add("Weibo");
        System.out.println(sites);
    }
}
```

The output is: [Google, Amazon, Taobao, Weibo]

LinkedList class

Add an element at the beginning of the list:

```
import java.util.LinkedList;

public class LinkedListTest {
    public static void main(String[] args) {
        LinkedList<String> sites = new LinkedList<String>();
        sites.add("Google");
        sites.add("Amazon");
        sites.add("Taobao");
        // Use addFirst() to add an element to the header
        sites.addFirst("Wiki");
        System.out.println(sites);
    }
}
```

The output is: [Wiki, Google, Amazon, Taobao]

LinkedList class

Add an element in the middle of the list:

```
import java.util.LinkedList;

public class LinkedListTest {
    public static void main(String[] args) {
        LinkedList<String> sites = new LinkedList<String>();
        sites.add("Google");
        sites.add("Amazon");
        sites.add("Taobao");
        // Use add (index, element) to add an element in the middle
        sites.add (1, "Wiki");
        System.out.println(sites);
    }
}
```

The output is: [Google, Wiki, Amazon, Taobao]

LinkedList class

Note: Is it LinkedList faster than ArrayList when adding elements in the middle of a list?

Generally speaking, since the bottom layer of ArrayList is an array, adding data needs to move the data b ehind it, while LinkedList uses a linked list, just move the pointer directly, so LinkedList should be faster.

However, the selection of the insertion position has a great impact on LinkedList, because LinkedList ne eds to move the pointer to the specified node before it can start insertion, once the insertion position is farther away, LinkedList needs to move the pointer step by step until it moves to the insertion position, so the larger t he insertion node value, the longer the LinkedList will take, because it takes time for the pointer to move. Arra yList is an array structure, and the position can be obtained directly according to the subscript, which saves th e time of finding specific nodes, so the impact on ArrayList is not particularly large.

Summary: Although there are the above situations, because ArrayList can use subscripts to directly obtai n data, ArrayList is generally selected when using queries, and LinkedList is more convenient when deleting a nd adding, so LinkedList is generally used more.

LinkedList class

Add an element at the end of the list:

```
import java.util.LinkedList;
public class LinkedListTest {
    public static void main(String[] args) {
        LinkedList<String> sites = new LinkedList<String>();
        sites.add("Google");
        sites.add("Amazon");
        sites.add("Taobao");
        // Use addLast() to add elements at the end
        sites.addLast("Wiki");
        System.out.println(sites);
    }
}
```

The output is: [Google, Amazon, Taobao,Wiki]

LinkedList class

Remove elements at the beginning of the list:

```
import java.util.LinkedList;
public class LinkedListTest {
    public static void main(String[] args) {
        LinkedList<String> sites = new LinkedList<String>();
        sites.add("Google");
        sites.add("Amazon");
        sites.add("Taobao");
       sites.add("Weibo");
        // Use removeFirst() to remove the header element
        sites.removeFirst();
        System.out.println(sites);
    }
}
```

The output is: [Amazon, Taobao,Weibo]

LinkedList class

Remove elements at the end of the list:

```
import java.util.LinkedList;
public class LinkedListTest {
    public static void main(String[] args) {
        LinkedList<String> sites = new LinkedList<String>();
        sites.add("Google");
        sites.add("Amazon");
        sites.add("Taobao");
       sites.add("Weibo");
        // Use removeLast() to remove the tail element
        sites.removeLast();
        System.out.println(sites);
    }
}
```

The output is: [Google,Amazon, Taobao]

LinkedList class

Get the element at the beginning of the list:
Use getFirst() to get the header element

Get the elements at the end of the list:
Use getLast() to get the tail element

LinkedList class

Iterate Elements: We can use for to iterate over elements in a list

```
import java.util.LinkedList;
public class LinkedListTest {
    public static void main(String[] args) {
        LinkedList<String> sites = new LinkedList<String>();
        sites.add("Google");
        sites.add("Amazon");
        sites.add("Taobao");
        sites.add("Weibo");
        for (int size = sites.size(), i = 0; i < size; i++) {
            System.out.println(sites.get(i));
        }
    }
}
```

The output is:

```
Google
Amazon
Taobao
Weibo
```

LinkedList class

Iterate on elements: You can also use for-each to iterate on elements:

```
import java.util.LinkedList;
public class LinkedListTest {
    public static void main(String[] args) {
        LinkedList<String> sites = new LinkedList<String>();
        sites.add("Google");
        sites.add("Amazon");
        sites.add("Taobao");
        sites.add("Weibo");
        for (String i : sites) {
            System.out.println(i);
        }
    }
}
```

The output is:

```
Google
Amazon
Taobao
Weibo
```

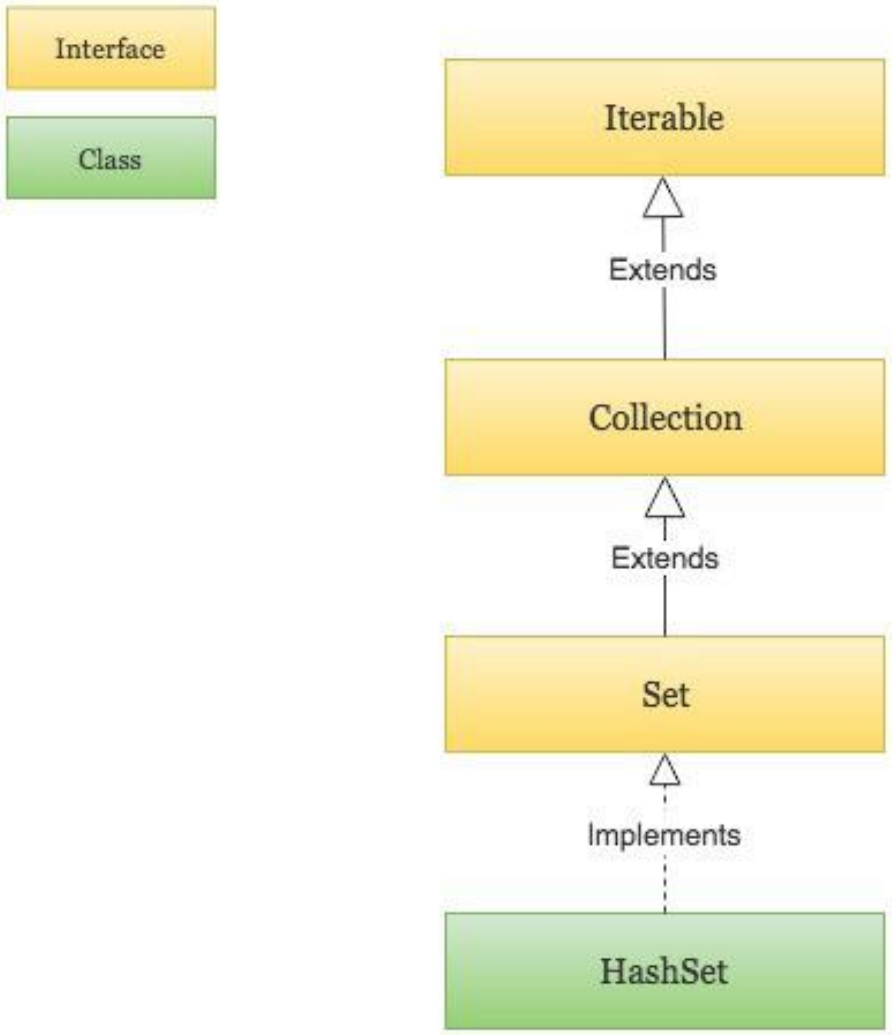
Collections and Strategies, Iterator Patterns

- **Overview of collection classes**
- **List interface and its standard implementation**
- **ArrayList and LinkedList**
- **Set interfaces with Map**
- **Strategy pattern**
- **Iterator pattern**

Set collection

- The objects in the Set collection are not sorted in a specific way, but simply add objects to the collection, but the Set collection cannot contain duplicate objects.
- The Set collection consists of the Set interface and the implementation class of the Set interface. The Set interface inherits the Collection interface and therefore contains all the methods of the Collection interface.

Java HashSet



HashSet class

The HashSet class is located in the java.util package and needs to be introduced before using it, with the following syntax format:

```
import java.util.HashSet;

HashSet<E> sites = new HashSet<E>();
```

E: Generic data type, used to set objectName data type, can only be a reference data type.

HashSet class

Add elements: The HashSet class provides many useful ways for classes, and adding elem ents can be done using the add() method:

```
import java.util.HashSet;
public class HashsetTest {
    public static void main(String[] args) {
    HashSet<String> sites = new HashSet<String>();
        sites.add("Google");
        sites.add("Amazon");
        sites.add("Taobao");
        sites.add("Zhihu");
        sites.add("Amazon");
        System.out.println(sites);
    }
}
```

The output is: [Google, Amazon, Taobao, Zhihu]

HashSet class

Determining the existence of an element: We can use the contains() method to determine whether an element exists in a collection

```
import java.util.HashSet;
public class HashsetTest {
    public static void main(String[] args) {
    HashSet<String> sites = new HashSet<String>();
        sites.add("Google");
        sites.add("Amazon");
        sites.add("Taobao");
        sites.add("Zhihu");
        sites.add("Amazon");  // Duplicate elements are not added
        System.out.println(sites.contains("Taobao"));
    }
}
```

The output is: true

HashSet class

Remove an element:
We can use the remove() method to remove elements from a collection:

```
sites.remove("Taobao");  // Delete an element, successful returns true, otherwise false
```

Deleting all elements in a collection can be done using the clear method:

```
sites.clear();
```

HashSet class

Calculate the size:

If you want to calculate the number of elements in a HashSet, you can use the size() method:

```
sites.size()
```

HashSet class

Iterate HashSet: You can use for-each to iterate over elements in a Hashset.

```
import java.util.HashSet;
public class HashsetTest {
   public static void main(String[] args) {
   HashSet<String> sites = new HashSet<String>();
      sites.add("Google");
      sites.add("Amazon");
      sites.add("Taobao");
      sites.add("Zhihu");
      sites.add("Amazon");  // Duplicate elements are not added
      for (String i : sites) {
         System.out.println(i);
      }
   }
}
```

The output is:

```
Google
Amazon
Taobao
Zhihu
```

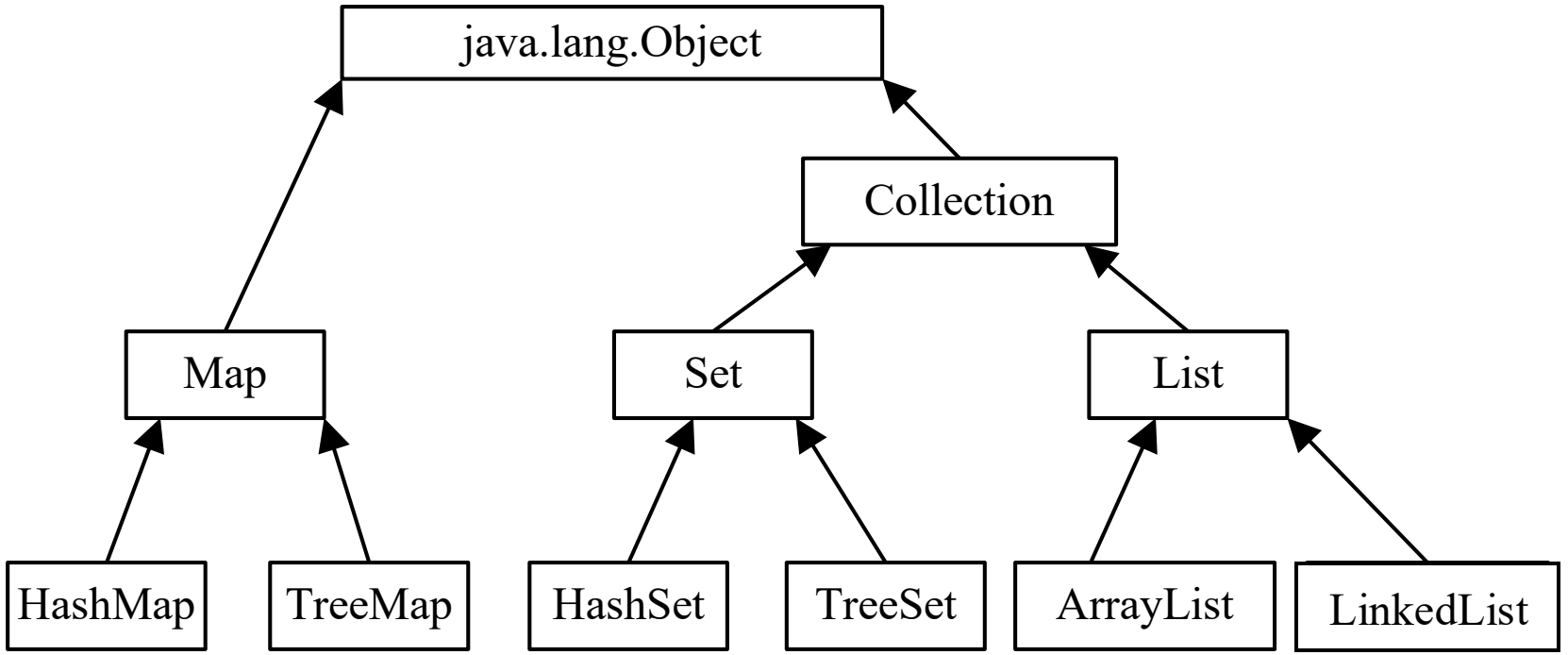
Map collection

- The Map interface provides objects that map key to value. A mapping cannot contain duplicate keys, each key can only be mapped to a maximum of one value. The Map interface also provides common methods for collections, as shown in the table below.

Method	Description
put(K key, V value)	Associates the specified value with the specified key. If the key already exists, its value is replaced.
get(Object key)	Returns the value to which the specified key is mapped, or null if the map contains no mapping for the key.
containsKey(Object key)	Returns true if this map contains a mapping for the specified key.
containsValue(Object value)	Returns true if this map maps one or more keys to the specified value.
keySet()	Returns a Set view of all the keys contained in this map.
values()	Returns a Collection view of all the values contained in this map.

What is a collection?

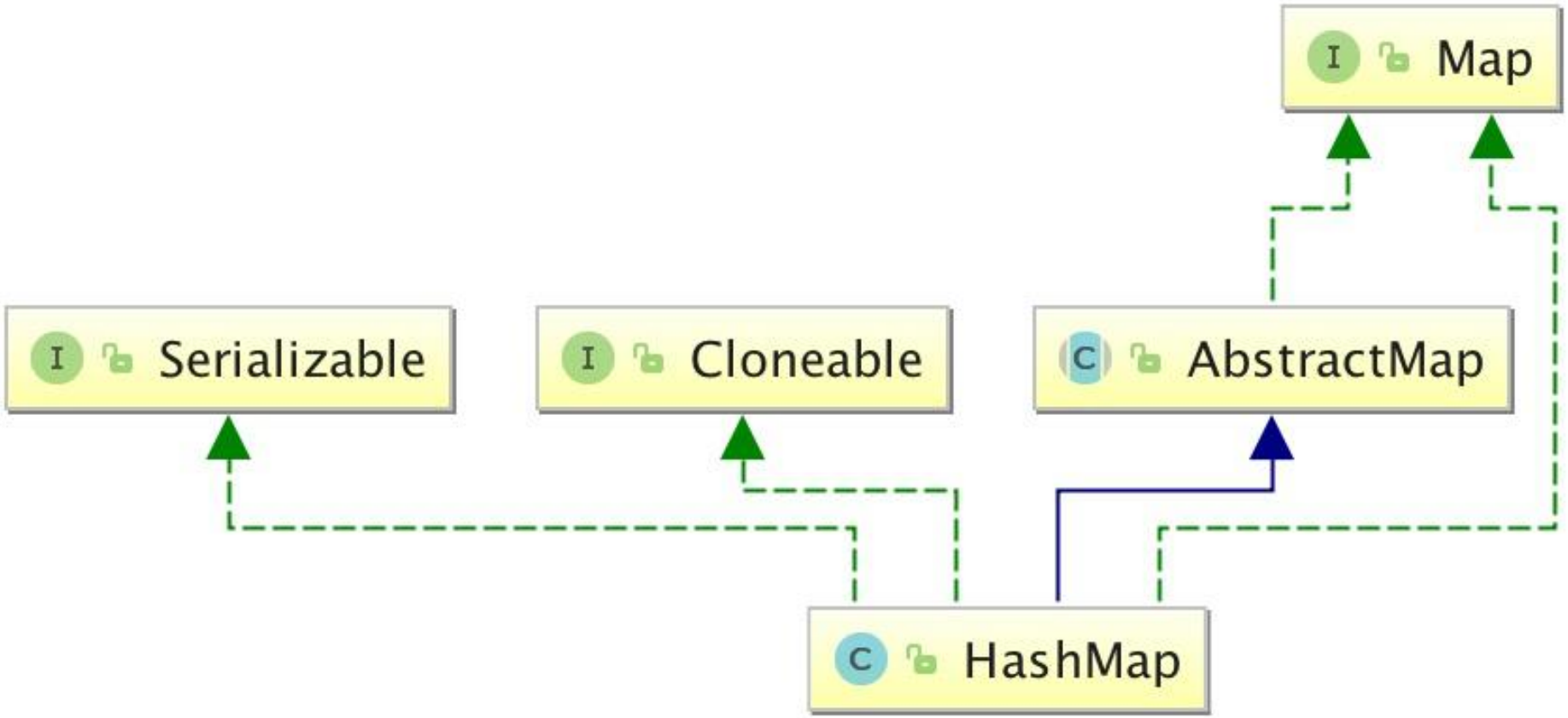
- The most basic interface is the Collection interface, which defines the basic methods for manipulating data.
- Commonly used collections include List collection, Set collection, and Map collection, where List and Set implement the Collection interface. Each interface also provides different implementation classes.



HashMap

- A HashMap is a hash list that stores key-value mappings.
- HashMap implements the Map interface, which stores data according to the HashCode value of the key, has a fast access speed, allows up to one recorded key to be null.
- HashMaps are out of order, meaning they don't record the order in which they are inserted.
- HashMap inherits from AbstractMap and implements Map, Cloneable, and java.io.Serializable interfaces.

HashMap



HashMap

The key and value types of HashMap can be the same or different, either key and value of the String type, or the key and value of the String type of the integer.

The HashMap class is located in the java.util package, and it needs to be introduced before use, and the syntax format is as follows:

```
import java.util.HashMap;
```

In the following example, we create a HashMap object Sites, an integer with a key and a value of type String:

```
HashMap<Integer, String> Sites = new HashMap<Integer, String>();
```

HashMap

Adding elements: The HashMap class provides a number of useful ways to add key-value pairs using the put() method:

```
import java.util.HashMap;
public class HashMapTest {
   public static void main(String[] args) {
      // create HashMap object Sites
      HashMap<Integer, String> Sites = new HashMap<Integer, String>();
      // Add key-value pairs
      Sites.put(1, "Google");
      Sites.put(2, “Amazon");
      Sites.put(3, "Taobao");
      Sites.put(4, "Zhihu");
      System.out.println(Sites);
   }
}
```

The output is: {1=Google, 2=Amazon, 3=Taobao, 4=Zhihu}

HashMap

Accessing elements: We can use the get(key) method to get the value corresponding to the key:

```
import java.util.HashMap;
public class HashMapTest {
   public static void main(String[] args) {
      // create HashMap object Sites
      HashMap<Integer, String> Sites = new HashMap<Integer, String>();
      // Add key-value pairs
      Sites.put(1, "Google");
      Sites.put(2, “Amazon");
      Sites.put(3, "Taobao");
      Sites.put(4, "Zhihu");
      System.out.println(Sites.get(3));
   }
}
```

The output is: Taobao

HashMap

Remove an element:

We can use the remove(key) method to remove the key-value pair corresponding to the key:

Sites.remove(4);

Deleting all elements in a collection can be done using the clear method:

sites.clear();

HashMap

Calculate the size:

If you want to count the number of elements in a HashMap, you can use the size() method:

sites.size()

HashMap

Iterate HashMap: You can use for-each to iterate over elements in a HashMap.

```
import java.util.HashMap;
public class HashMapTest {
   public static void main(String[] args) {
      // create HashMap object Sites
      HashMap<Integer, String> Sites = new HashMap<Integer, String>();
      // Add key-value pairs
      Sites.put(1, "Google");
      Sites.put(2, “Amazon");
      Sites.put(3, "Taobao");
      Sites.put(4, "Zhihu");
```

HashMap

Iterate HashMap: You can use for-each to iterate over elements in a HashMap.

```
        // Output key and value (if you just want to get the key, you can use the keySet() method)
        for (Integer i : Sites.keySet()) {
            System.out.println("key: " + i + " value: " + Sites.get(i));
        }
        // Output value value (if you just want to get value, you can use the values() method) method)
        for(String value: Sites.values()) {
          System.out.print(value + ", ");
        }
    }
}
```

The output is:

```
key: 1 value: Google
key: 2 value: Amazon
key: 3 value: Taobao
key: 4 value: Zhihu
Google, Amazon, Taobao, Zhihu,
```

Collections and Strategy, Iterator Patterns

- **Overview of collection classes**
- **List interface and its standard implementation**
- **ArrayList and LinkedList**
- **Set interfaces with Map**
- **Strategy pattern**
- **Iterator pattern**

Strategy patterns

- Strategy Pattern

Using multiple if-else statements can make the code complex and difficult to maintain in situations where multiple algorithms are similar, while strategy patterns allow policies to change as objects change.

- Many times we can have multiple strategies to achieve our goals
  - The way to travel, choose to ride a bicycle or take a car, each of which is a strategy
  - ...

Strategy patterns

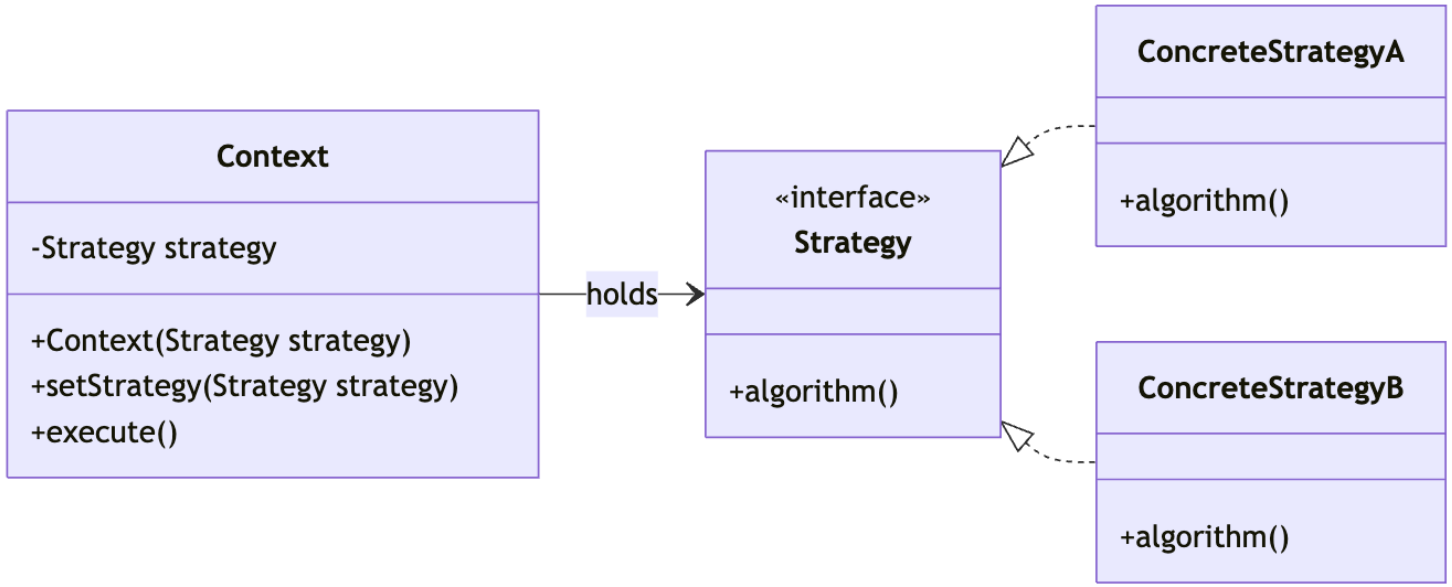
How do you separate the algorithm from the object so that the algorithm can change independently of the customer using it?

Define a series of algorithms, encapsulate each algorithm, and make them interchangeable with each other, so that the algorithm can change independently of the customer using it. And move the logical judgment to the client (i.e. let the client decide what specific policy to use in what situation).

Strategy Pattern

The strategy mode involves three roles:

- Abstract Strategy Role: This is an abstract role, usually implemented by an interface or abstract class. This role gives all the interfaces required for specific policy classes.
- ConcreteStrategy role: Wraps the relevant algorithm or behavior.
- Context role: Holds a reference to a Strategy.



Strategy Pattern

Example: Suppose you want to design an e-commerce website that sells books, and this website may offer a promotional discount of 20% per book to all premium members; 10% promotional discount per book for mid-level members; There are no discounts for junior members.

According to the description, discounts are carried out according to one of several algorithms:

- Algorithm 1: No discount for junior members.
- Algorithm 2: Offer a 10% promotional discount to mid-level members.
- Algorithm 3: Offer a 20% promotional discount for premium members.

Strategy Pattern

Abstract discount class:

```
public interface MemberStrategy {
    /**
     * Calculate the price of the book
     * @param booksPrice    The original price of the book
     * @return    Calculate the discounted price
     */
    public double calcPrice(double booksPrice);
}
```

Primary Member Discounts:

```
public class PrimaryMemberStrategy implements MemberStrategy {
    @Override
    public double calcPrice(double booksPrice) {
        System.out.println("There are no discounts for junior members");
        return booksPrice;
    }
}
```

Strategy Pattern

Intermediate Membership Discounts:

```
public class IntermediateMemberStrategy implements MemberStrategy {
    @Override
    public double calcPrice(double booksPrice) {
        System.out.println("10% discount for intermediate members");
        return booksPrice * 0.9;
    }
}
```

Advanced Membership Discounts:

```
public class AdvancedMemberStrategy implements MemberStrategy {
    @Override
    public double calcPrice(double booksPrice) {
        System.out.println("20% discount for premium members");
        return booksPrice * 0.8;
    }
}
```

Strategy Pattern

Websites:

```
public class Network { //Hold a specific policy object
        private MemberStrategy strategy;
        /**
        * constructor, pass in a specific policy object
        * @param strategy Specific policy objects
        */
        public Network(MemberStrategy strategy){
                this.strategy = strategy;
        }
        /**
          *Calculate the price of the book
        * @param booksPrice The original price of the book
        * @return Calculate the discounted price
        */
        public double quote(double booksPrice){
           return this.strategy.calcPrice(booksPrice);
        }
}
```

Strategy Pattern

Client class:

```
public class Client { public static void main(String[] args) {
        //Select and create the policy object that you want to use
        MemberStrategy strategy = new AdvancedMemberStrategy();
        //Create an environment
        Network network = new Network(strategy);
        //Calculate the price
        double quote = network.quote(300);
        System.out.println("The final price of the book is:" + quote); }
}
```

Strategy Pattern

Review:

- As you can see from the example above, the policy pattern encapsulates the algorithm
- It is convenient to insert new algorithms into existing systems, and old algorithms to "retire" from the system to achieve method replacement
- Policy patterns do not determine when and which algorithm to use. It is up to the client to decide what algorithm to use in what situation.

Strategy Pattern

Note:

- The focus of the policy pattern: The focus of the policy pattern is not how to implement the algorithms, b ut how to organize and call these algorithms, so as to make the program structure more flexible, have bett er maintainability and scalability.
- Equality of algorithms: A major feature of the strategy model is the equality of each strategy algorithm. F or a series of specific strategy algorithms, everyone's status is exactly the same, and it is precisely becaus e of this equality that the algorithms can replace each other. All strategy algorithms are independent of ea ch other in implementation and do not depend on each other.

Strategy Pattern

Note:

- Uniqueness of runtime policies: During operation, the policy mode can only use one specific policy implem entation object at each moment, although it can dynamically switch between different policy implementation s, but only one can be used at the same time.
- Public behavior: It is often seen that all specific strategies have some public behavior. At this time, these pu blic behaviors should be placed in the common abstract strategy role Strategy class. Of course, the abstract p olicy role must be implemented with Java abstract classes, not interfaces.

Strategy Pattern

Advantages：

- It provides an alternative to inheritance while maintaining the advantages of inheritance (code reuse) and being more flexible than inheritance (algorithmically independent and can be exten ded arbitrarily).
- It separates the logic of which algorithm to adopt or which behavior to take from the algorithm itself, avoids the use of multi-conditional transfer statements in the program, and makes the sy stem more flexible and easy to scale.
- Adhere to most of the design principles, high cohesion, low coupling.

Strategy Pattern

Disadvantages：

- The client must know all the policy classes and decide for themselves which policy class to us e. This means that the client must understand the difference between these algorithms in order to choose the appropriate algorithm class at the right time. In other words, the policy pattern is only applicable when the client knows the algorithm or behavior.
- Since the policy pattern encapsulates each specific policy implementation into a class, if there are many alternative strategies, the number of objects will be considerable.

Strategy Pattern

Summary of the strategy model:

element	description
Pattern Name	**Strategy Pattern**
Intent	Avoid the complexity and difficulty of maintaining multiple if-else statements when there are multiple algorithms that are similar.
Problem	How to separate the algorithm from the object so that the algorithm can change independently of the customer using it.
Solution	Define a series of algorithms, encapsulate each algorithm, and make them interchangeable with each other, so that the algorithm can change independently of the customer using it.
Consequence	Advantages: 1. The algorithm can be switched freely. 2. Avoid using multiple conditions to judge. 3. Good scalability. Disadvantages: 1. There will be more strategy classes. 2. All strategies need to be exposed.

Collections and Strategies, Iterator Patterns

- **Overview of collection classes**
- **List interface and its standard implementation**
- **ArrayList and LinkedList**
- **Set interfaces with Map**
- **Strategy pattern**
- **Iterator pattern**

Iterator Pattern

- **Iterator Pattern**

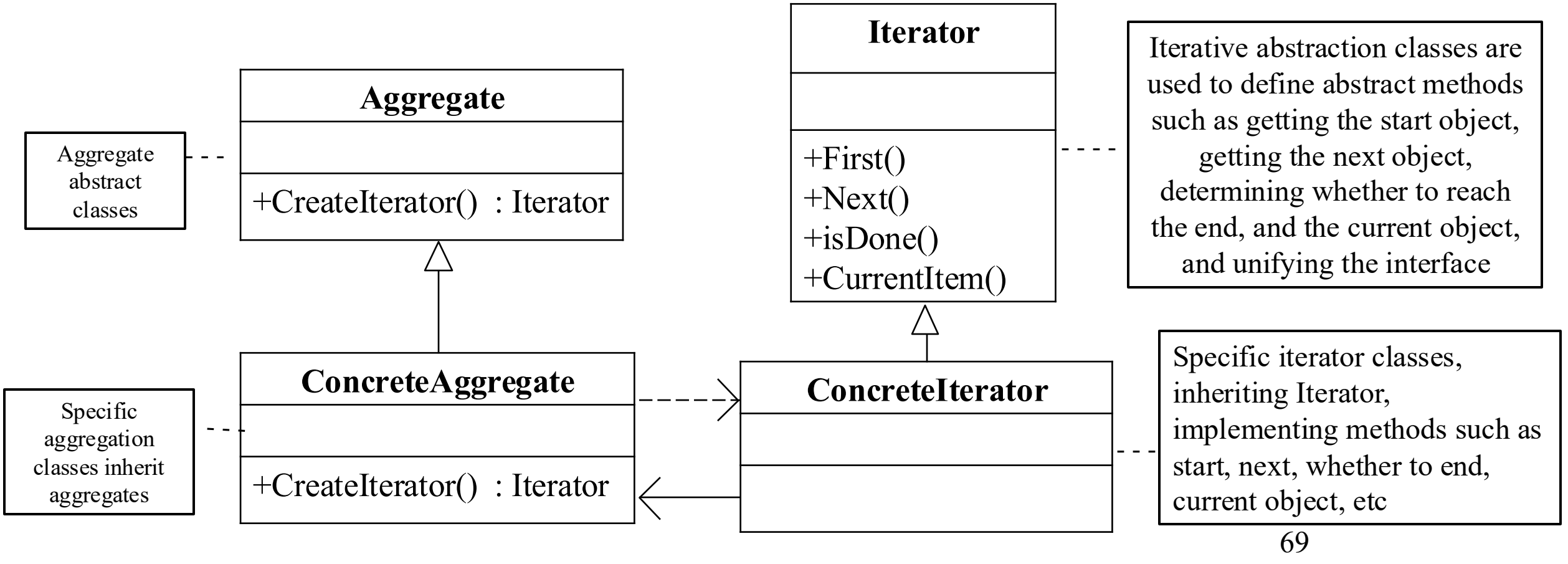
Inserting an iterator between the customer access class and the aggregate class separates the aggregate object from its traversal behavior, hides its internal details from the customer, and satisfies the "single duty principle" and "open and close principle".

- Iterator patterns are widely used in life
  - The conveyor belt in the logistics system, no matter what items are transmitted, is packaged into boxes and has a unified QR code, which only needs to be checked one by one when distributing.
  - When taking transportation, they all swipe their cards or face to enter the station, and they do not need to care about whether they are male or female, disabled or normal people.
  - ...

Iterator Pattern

How do I separate an aggregate object from its traversal behavior?

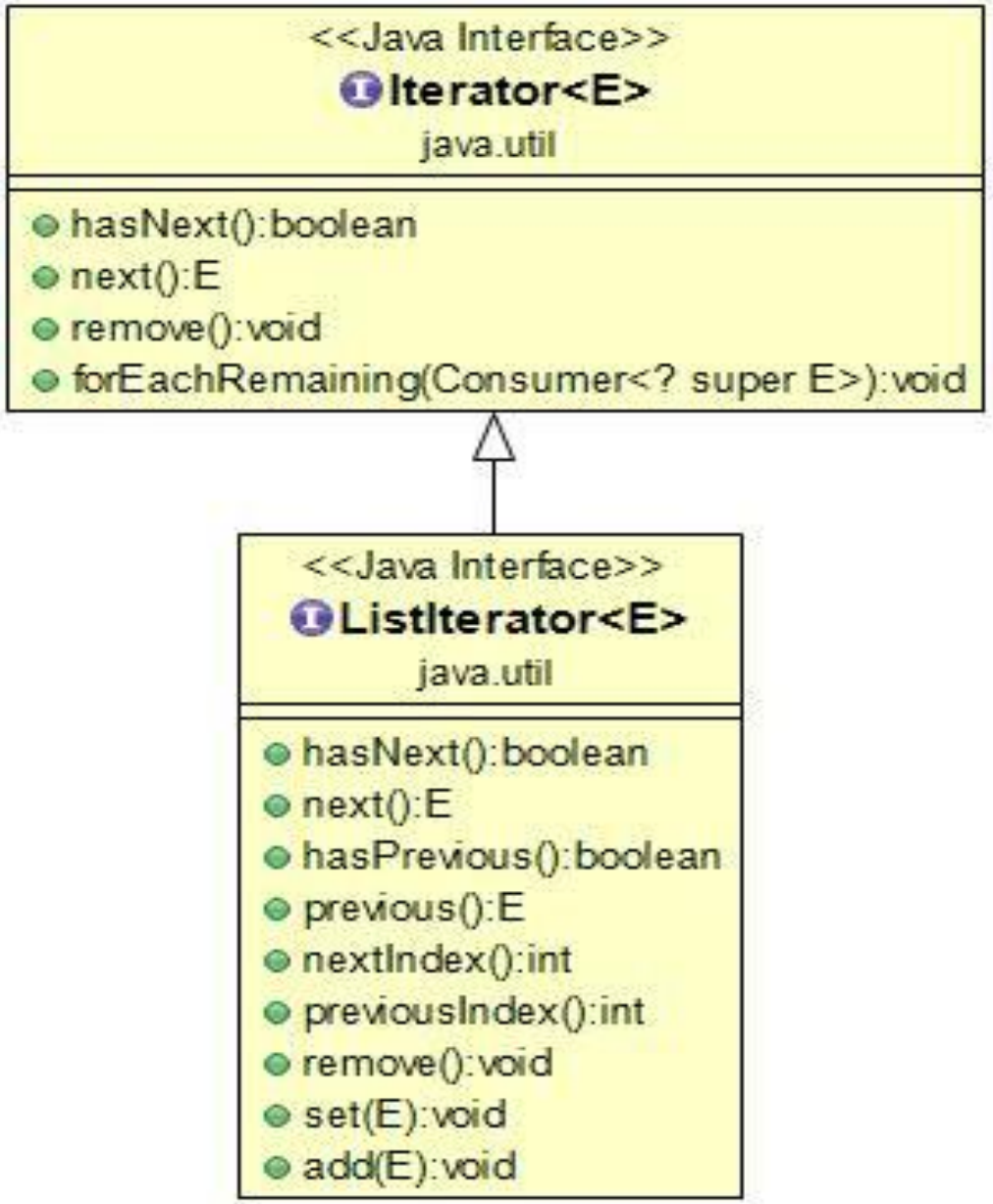
Iterator Pattern Diagram



Iterator Pattern

- A Java Iterator is not a collection, it is a method for accessing collections that can be used to iterate over c ollections such as ArrayList and HashSet.
- Iterator is the simplest implementation of Java iterators, and ListIterator is an interface in the Collection A PI that extends the Iterator interface.

Iterator Pattern



Iterator Pattern

- The three basic operations of iterator it are next, hasNext, and remove.
- Calling it.next() returns the next element of the iterator and updates the state of the iterator.
- Calling it.hasNext() is used to detect if there are still elements in the collection.
- Call it.remove() to remove the element returned by the iterator.
- The Iterator class is located in the java.util package and needs to be introduced before use, with the following syntax format:

```
import java.util.Iterator;
```

Iterator Pattern

```
import java.util.ArrayList;
import java.util.Iterator;
public class IteratorTest {
    public static void main(String[] args) {
        // Create a collection
        ArrayList<String> sites = new ArrayList<String>();
        sites.add("Google");
        sites.add("Amazon");
        sites.add("Taobao");
        sites.add("Zhihu");
        // Get the iterator
        Iterator<String> it = sites.iterator();
        // Output the first element in the collection
        System.out.println(it.next());
    }
}
```

The output is: Google

Iterator Pattern

Loop through collection elements: The easiest way to have the iterator it return all the elements in the collection one by one is to use a while loop:

```
while(it.hasNext()) {
        System.out.println(it.next());
}
```

The output is:

```
Google
Amazon
Taobao
Weibo
```

Thinking questions

- How many for loops do you have?

```
for(int i = 0; i < myCollection.size(); i++) {
        Item element = myCollection.get(i);
        ...
}
```

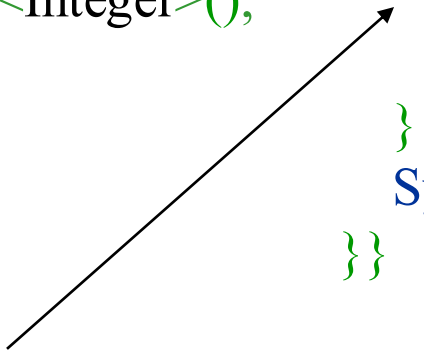
```
for(Item element : myCollection) { ... }
```

```
Iterator<Item> iterator = myCollection.iterator();
while(iterator.hasNext()) {
 Item element = iterator.next();
 ...
 }
```

Iterator Pattern

Delete Element: To delete an element in a collection, you can use the remove() method.
In the following example, we remove elements that are less than 10 in a collection:

```
import java.util.ArrayList;
import java.util.Iterator;
public class IteratorTest {
    public static void main(String[] args) {
        ArrayList<Integer> numbers = new ArrayList<Integer>();
        numbers.add(12);
        numbers.add(8);
        numbers.add(2);
        numbers.add(23);
        Iterator<Integer> it = numbers.iterator();
        while(it.hasNext()) {
            Integer i = it.next();
            if(i < 10) {
              // Delete elements that are less than 10
              it.remove();
            }
        }
        System.out.println(numbers);
    }}
```



The output is: [12, 23]

Question

Which of the following descriptions of the for loop vs. iterator is incorrect?

A. The for loop can support quickly fetching elements at a specified location

B. Iterator uses a sequential access approach

C. Iterator is not suitable for LinkedList

D. ArrayList is suitable for using for loops

submit

Question

Which of the following descriptions of the for loop vs. iterator is incorrect?

A. The for loop can support quickly fetching elements at a specified location

B. Iterator uses a sequential access approach

C. Iterator is not suitable for LinkedList

D. ArrayList is suitable for using for loops

submit

Iterator Pattern

Iterator Pattern Summary:

element	description
Pattern Name	Iterator Pattern
Intent	Provides a way to access individual elements of an aggregate object sequentially without exposing the internal representation of the object.
Problem	1. Access the content of an aggregate object without exposing its internal representation. 2. It is necessary to provide multiple traversal methods for the aggregation object. 3. Provide a unified interface for traversing different aggregate structures.
Solution	Give the responsibility of moving between elements to the iterator, not the aggregate object.
Consequence	Pros: 1) It supports traversing an aggregate object in different ways. 2) Iterators simplify aggregation classes. 3) There can be multiple traversal on the same aggregate. 4) In iterator mode, it is convenient to add new aggregate classes and iterator classes without modifying the original code.

Iterator discussion

- An iterator is a pattern that separates the traversal behavior of a data structure of a sequence type from the object being traversed, i.e. we don't need to care about what the underlying structure of the sequence looks like
  - Case: If you use Iterator to traverse the elements in the collection, once you stop using List and use Set to organize the data, the code for traversing the elements does not need to be modified; If you use for to traverse, then all the algorithms that traverse this set will have to be adjusted accordingly, because the List is ordered, the Set is unordered, the structure is different, and their access algorithms are also different
- When looping, the Iterator can be used to delete the element; and cannot remove elements in a for loop

Comparison

- Advantages
  - Supports multiple collection traversals
  - The packaging is good, and the user only needs to get the iterator to iterate, and there is no need to worry about the traversal algorithm
- Disadvantages
  - For simpler traversal (array or ordered list), using iterator traversal is cumbersome and inefficient
  - Using iterators is more suitable for collections that are implemented in the form of linked lists at the bottom level

Collections and Strategy, Iterator Patterns

- **Overview of collection classes**
- **List interface and its standard implementation**
- **ArrayList and LinkedList**
- **Set interfaces with Map**
- **Strategy pattern**
- **Iterator pattern**